Titotto Paolo 895227

REPORT CHALLENGE 3

Sommario

REPORT CHALLENGE 3 1
IDEA RISOLUTIVA 2
CODICE RISOLUTIVO 4
OUTPUT DEL PROGRAMMA E STAMPA DEL PIN 5
VERIFICA DEGLI OTP 5

Titotto Paolo 895227

IDEA RISOLUTIVA

L'idea risolutiva usata per questo challenge è la seguente:

- È stata creata una funzione generate_otp che prende due parametri: i timestamp e i pin generati. Calcola l'hash del pin e ne genera l'otp prendendo le prime 6 cifre e non 8 come il pin.

```
def generate_otp(pin, timestamp):
    size = 6
    otp = ''
    h = hashlib.sha256((pin + timestamp).encode()).digest()
    for i in range(size):
        digit = h[i] % 10
        otp += str(digit)
    return otp
```

- Viene creata una funzione find_pin che prende come parametro un dizionario otp_dict che contiene come chiave i timestamp e come valore gli otp generati.
  Il metodo inizia con un ciclo for con un range di 100000000 che sono tutte le possibili combinazioni di pin generabili da 00000000 a 99999999. La riga successiva converte il pin generato in stringa e utilizza la funzione zfill(8) in modo da creare sempre pin della lunghezza corretta aggiungendo 0 davanti in caso di necessità.

```
for pin in range(100000000):
    pin_str = str(pin).zfill(8)  # Ensure it's 8 digits long
```

  La variabile generated_otps crea un dizionario con chiave i timestamp e come valore l'otp generato dalla funzione precedente in base al pin e la data associata. Tutto questo viene svolto per ogni chiave del dizionario otp_dict

```
# Generate OTPs for current PIN and put them in a dictionary with
the timestap key
generated_otps = {timestamp: generate_otp(pin_str, timestamp) for
timestamp in otp_dict}
```

  Si arriva dunque alla parte conclusive della funzione, quella dove viene fatto il controllo degli otp. Viene fatto l'if su tutti gli otp generati ad uno specifico timestamp presente nel dizionario (richiamato dalla funzione generate_otp[timestamp]) controllando che sia uguale all'otp di otp_dict. In caso affermativo restituisce il pin altrimenti None..

```
if all(generated_otps[timestamp] == otp for timestamp, otp in
otp_dict.items()):
        return pin_str
return None
```

- Inoltre viene fatta una funzione main dove viene creato otp_dict con tutti i rispettivi valori e chiavi. Viene invocata la funzione find_pin sul dictionary appena istanziato e attraverso un controllo verifica se il pin è stato trovato ritornandolo oppure stampando il fatto che non è riuscito a scovarlo.

```
def main():
    otp_dict = {
        '2020-4-8-21:45': '754104',
        '2020-4-8-21:46': '353471',
        '2020-4-8-21:47': '084633'
    }

    print("Trying to find the secret PIN...")
    secret_pin = find_pin(otp_dict)
    if secret_pin:
        print(f'The secret PIN is: {secret_pin}')
    else:
        print('Could not find the secret PIN.')
```

CODICE RISOLUTIVO

Nelle righe che seguono è presente il codice per interno con relative commenti:

```
import hashlib


def generate_otp(pin, timestamp):
    size = 6
    otp = ''
    h = hashlib.sha256((pin + timestamp).encode()).digest()
    for i in range(size):
        digit = h[i] % 10
        otp += str(digit)
    return otp


def find_pin(otp_dict):
    for pin in range(100000000):
        pin_str = str(pin).zfill(8)  # Ensure it's 8 digits long
        # Generate OTPs for current PIN and put them in a dictionary with the
timestap key
        generated_otps = {timestamp: generate_otp(pin_str, timestamp) for
timestamp in otp_dict}
        # Print the current PIN and the generated OTPs
        print(f"Trying PIN: {pin_str}, Generated OTPs: {generated_otps}")
        #check all the opts generated at the specific timestamp with the the otp
in the dictionary
        if all(generated_otps[timestamp] == otp for timestamp, otp in
otp_dict.items()):
            return pin_str
    return None


def main():
    otp_dict = {
        '2020-4-8-21:45': '754104',
        '2020-4-8-21:46': '353471',
        '2020-4-8-21:47': '084633'
    }

    print("Trying to find the secret PIN...")
    secret_pin = find_pin(otp_dict)
    if secret_pin:
        print(f'The secret PIN is: {secret_pin}')
    else:
        print('Could not find the secret PIN.')


if __name__ == "__main__":
    main()
```

Titotto Paolo 895227

OUTPUT DEL PROGRAMMA E STAMPA DEL PIN

Qui sotto sono presenti alcune stampe dei pin generati e il pin segreto trovato

```
Trying PIN: 41122814, Generated OTPs: {'2020-4-8-21:45': '435108', '2020-4-8-21:46': '767813', '2020-4-8-21:47': '317741'}
Trying PIN: 41122815, Generated OTPs: {'2020-4-8-21:45': '463099', '2020-4-8-21:46': '622013', '2020-4-8-21:47': '385421'}
Trying PIN: 41122816, Generated OTPs: {'2020-4-8-21:45': '544941', '2020-4-8-21:46': '935888', '2020-4-8-21:47': '635857'}
Trying PIN: 41122817, Generated OTPs: {'2020-4-8-21:45': '689966', '2020-4-8-21:46': '471045', '2020-4-8-21:47': '463601'}
Trying PIN: 41122818, Generated OTPs: {'2020-4-8-21:45': '995113', '2020-4-8-21:46': '114763', '2020-4-8-21:47': '377534'}
Trying PIN: 41122819, Generated OTPs: {'2020-4-8-21:45': '419299', '2020-4-8-21:46': '113591', '2020-4-8-21:47': '381933'}
Trying PIN: 41122820, Generated OTPs: {'2020-4-8-21:45': '967364', '2020-4-8-21:46': '328757', '2020-4-8-21:47': '356954'}
Trying PIN: 41122821, Generated OTPs: {'2020-4-8-21:45': '003189', '2020-4-8-21:46': '181523', '2020-4-8-21:47': '253318'}
Trying PIN: 41122822, Generated OTPs: {'2020-4-8-21:45': '104639', '2020-4-8-21:46': '745265', '2020-4-8-21:47': '847843'}
Trying PIN: 41122823, Generated OTPs: {'2020-4-8-21:45': '776361', '2020-4-8-21:46': '645316', '2020-4-8-21:47': '995183'}
Trying PIN: 41131320, Generated OTPs: {'2020-4-8-21:45': '871460', '2020-4-8-21:46': '178401', '2020-4-8-21:47': '377636'}
Trying PIN: 41131321, Generated OTPs: {'2020-4-8-21:45': '541674', '2020-4-8-21:46': '947993', '2020-4-8-21:47': '712276'}
Trying PIN: 41131322, Generated OTPs: {'2020-4-8-21:45': '394101', '2020-4-8-21:46': '186052', '2020-4-8-21:47': '847071'}
Trying PIN: 41131323, Generated OTPs: {'2020-4-8-21:45': '750199', '2020-4-8-21:46': '569625', '2020-4-8-21:47': '310858'}
Trying PIN: 41131324, Generated OTPs: {'2020-4-8-21:45': '697049', '2020-4-8-21:46': '112394', '2020-4-8-21:47': '655774'}
Trying PIN: 41131325, Generated OTPs: {'2020-4-8-21:45': '764487', '2020-4-8-21:46': '742992', '2020-4-8-21:47': '128466'}
Trying PIN: 41131326, Generated OTPs: {'2020-4-8-21:45': '110939', '2020-4-8-21:46': '258341', '2020-4-8-21:47': '181305'}
Trying PIN: 41131327, Generated OTPs: {'2020-4-8-21:45': '790452', '2020-4-8-21:46': '180926', '2020-4-8-21:47': '667476'}
Trying PIN: 41131328, Generated OTPs: {'2020-4-8-21:45': '215545', '2020-4-8-21:46': '932219', '2020-4-8-21:47': '438192'}
Trying PIN: 41131329, Generated OTPs: {'2020-4-8-21:45': '005702', '2020-4-8-21:46': '708387', '2020-4-8-21:47': '383657'}
Trying PIN: 41131330, Generated OTPs: {'2020-4-8-21:45': '503645', '2020-4-8-21:46': '857058', '2020-4-8-21:47': '490596'}
Trying PIN: 41131331, Generated OTPs: {'2020-4-8-21:45': '526030', '2020-4-8-21:46': '434699', '2020-4-8-21:47': '393887'}
Trying PIN: 41131332, Generated OTPs: {'2020-4-8-21:45': '115887', '2020-4-8-21:46': '951350', '2020-4-8-21:47': '303775'}
Trying PIN: 41131333, Generated OTPs: {'2020-4-8-21:45': '565529', '2020-4-8-21:46': '979589', '2020-4-8-21:47': '746813'}
Trying PIN: 41131334, Generated OTPs: {'2020-4-8-21:45': '281508', '2020-4-8-21:46': '363703', '2020-4-8-21:47': '952522'}
Trying PIN: 41131335, Generated OTPs: {'2020-4-8-21:45': '170824', '2020-4-8-21:46': '805451', '2020-4-8-21:47': '734745'}
Trying PIN: 41131336, Generated OTPs: {'2020-4-8-21:45': '662022', '2020-4-8-21:46': '097538', '2020-4-8-21:47': '089685'}
Trying PIN: 41131337, Generated OTPs: {'2020-4-8-21:45': '754104', '2020-4-8-21:46': '353471', '2020-4-8-21:47': '084633'}
The secret PIN is: 41131337
```

VERIFICA DEGLI OTP

Per verificare la correttezza del pin trovato è stata copiata in un altro file la funzione generate_otp e istanziato un main che richiamava la funzione e come parametri prende il pin e i timestamp relativi.

```python
def generate_otp(pin, timestamp):
  size = 6
  otp = ''
  h = hashlib.sha256((pin + timestamp).encode()).digest()
  for i in range(size):
    digit = h[i] % 10
    otp += str(digit)
  return otp

if __name__ == '__main__':
  print(generate_otp('41131337', '2020-4-8-21:45'))
  print(generate_otp('41131337', '2020-4-8-21:46'))
  print(generate_otp('41131337', '2020-4-8-21:47'))
```

```
754104
353471
084633
```